

Test matrix, mutation testing, and benchmark
suite

Test matrix and coverage

The v0.5.0 Python implementation ships a test suite that, on the canonical `uv run pytest tests/ --cov=py/cogant run`, reports **2129 passing** tests with **86 skips** for optional dependencies (Rust toolchain, matplotlib, tree-sitter language grammars, PNG rasterization), plus **2 expected xfail** and **1 xpass** case. End-to-end runtime is on the order of four minutes on a 2024-class Apple-silicon workstation (**238 s** in the canonical run); the overall line coverage of `py/cogant/` is **83.42%** on that run, measured across **56628** lines in **179** source files (see `METRICS.yaml`).

Table 8. Python interpreter matrix.

Python version	pyproject.toml classifier	Status
3.11	Programming Language :: Python :: 3.11	supported (minimum version, requires-python = ">=3.11")
3.12	Programming	supported

Mutation testing

Mutation testing was performed on the algorithmic core modules (`gnn/matrices.py`, `translate/engine.py`, `markov/blanket.py`, `statespace/compiler.py`, `static/dataflow.py`). The canonical mutmut 3.5.0 runner was evaluated but rejected: on COGANT's test layout mutmut reported every one of the 403 auto-generated mutants on `matrices.py` as “no tests” because its v3 trampoline requires tests to import the mutated module through the `mutants/<path>` shadow tree, and the project's pytest configuration does not. Rather than ship a “no tests” score, the mutation analysis in

`../cogant/docs/evaluation/MUTATION_REPORT.md` is based on a **hand-picked set of fifteen semantic mutations** that target the algorithmic predicates, constants, and loop bounds of the above modules; each mutation was applied, the relevant pytest subset was rerun, and the mutation was reverted immediately. This is a more informative experiment than a green mutmut run because it documents exactly *which* invariants the tests enforce.

Table 10. Hand-curated mutation results on COGANT

Benchmark suite (shipped)

A reproducible benchmark harness lives at `../cogant/benchmarks/bench_suite.py` and writes its canonical results to `../cogant/benchmarks/results/`. The most recent run committed to the tree (`bench(p1.5)`: Rust build status + Python vs Rust benchmark results, then superseded by `bench(suite)`: reproducible 6-fixture benchmark harness with stage timing + memory + GNN stats) executed each fixture for three iterations on CPython 3.12.11 / macOS arm64 and recorded per-stage wall-clock time, peak memory, and the final GNN tensor shapes.

Table 11. Benchmark suite results (suite_20260409.md, three iterations per fixture, CPython 3.12.11).

Fixture	Wall-clock median (ms)	Wall-clock p95 (ms)	Nodes	Edges	Mappings	Peak mem- ory (MB)
calculator	32	35	12	25	11	0.0